Test an Ecto Query

It's simple to test code that uses database queries. We need to use a special test case that's provided to us by the Phoenix generator. This includes transactional test cases, which means that our test database is cleaned up after each test.

We'll test the ContactStore module because it has functions for retrieving and writing data. We will skip SmsMessageStore because it has many more functions. But the book's provided code (phone_app folder) has tests for the entire application.

Create test/phone_app/conversations/query/contact_store_test.exs and add the following code:

phone_app/test/phone_app/conversations/query/contact_store_test.exs

```
Line 1  defmodule PhoneApp.Conversations.Query.ContactStoreTest do
        use PhoneApp.DataCase, async: true

        alias PhoneApp.Conversations.Query.ContactStore
5
        describe "upsert_contact/1" do
          @incoming %{
            from: "111-222-3333",
            to: "999-888-7777",
10          direction: :incoming
          }

          @outgoing %{@incoming | direction: :outgoing}

15        test "a new contact is created, based on direction" do
            assert {:ok, contact} = ContactStore.upsert_contact(@incoming)
            assert contact.id
            assert contact.phone_number == "111-222-3333"

20          assert {:ok, contact2} = ContactStore.upsert_contact(@outgoing)
            assert contact2.id
            assert contact2.id != contact.id
            assert contact2.phone_number == "999-888-7777"
          end
25
          test "a contact with the same phone number is updated" do
            assert {:ok, contact} = ContactStore.upsert_contact(@incoming)
            assert {:ok, contact2} = ContactStore.upsert_contact(@incoming)

30          assert contact2.updated_at != contact.updated_at
            assert Map.delete(contact2, :updated_at) ==
                    Map.delete(contact, :updated_at)
          end
        end
```